Automated Software Testing: Quiz Study Notes

Prepared for Arham · 18 October 2026 · Weeks 1 to 3, with a preview of Weeks 4 to 7

Where these notes come from: your Week 1 slides (Katalon Recorder), Week 2 slides and handouts (Selenium, Maven, basic commands), Week 3 slides and handout (DOM and locators), the practice you did (Katalon runs and your own Java code), and your midterm notes PDF. Items marked **(extra)** are standard knowledge that is not written on your slides, so confirm them with your teacher before using them in an exam.

0. Progress map

Topic	Status
Week 1: manual vs automation, benefits, challenges	Covered
Week 1: Katalon Recorder (features, commands, locators, debugging, limitations)	Covered and practised
Week 2: Selenium, WebDriver, why Java	Covered
Week 2: Maven, pom.xml, TestNG annotations, FirstTest	Covered
Week 2: basic commands and your own First_Test code	Covered and reviewed
Week 3: DOM, elements, attributes, Inspect	Remaining (section 3)
Week 3: Java locators with By, findElements	Remaining (section 4)
Week 3: CSS and XPath	Started with Katalon, full notes in section 5
Weeks 4 to 7: interactions, waits, assertions, POM	Outline only (section 7)

1. Week 1: Introduction to Automated Testing

1.1 What is software testing?

Software testing means checking that an application does what its requirements say, by running it and looking for defects (bugs). The slides give three purposes: **find bugs, ensure quality, and verify that the software works as intended.**

Example (online shop checkout): can customers add products to the cart, can they place an order, and do they get proper error messages when required information is missing or invalid? Good testing checks what happens when things go wrong, not only when they go right.

1.2 Manual vs automation testing

- **Manual testing:** a person follows test cases by hand and acts like a real user. Example: compose and send a Gmail message, then check that it appears in the Sent folder.
- **Automation testing:** tools and scripts run the tests, compare **actual results** with **expected results**, and report the outcome. Example: a script searches Daraz, applies price and category filters, and checks that the products shown match. Tools named in the slides: Selenium, Cypress, Katalon Studio.
- **Core idea:** execute, compare actual vs expected, report. If actual equals expected the test **passes**, otherwise it **fails**.
- Automation suits repetitive, time-consuming and **regression** testing. Regression testing means re-checking old features after a change, to make sure nothing that used to work is now broken.

Aspect	Manual	Automation	Why
Execution	Human performs steps	Scripts perform steps	
Time	Slow for repeated tests	Fast for repeated tests	A script can run 100 checks in minutes
Cost	Low initial cost	High initial cost	Scripts, tools and skills must be built first
Flexibility	Easy to change and explore	Script needs updating	A human adapts instantly
Human error	More likely when repetitive	Fewer repetitive mistakes	Tired testers skip steps

Aspect	Manual	Automation	Why
Reusability	Repeated by hand each release	Same script reused each release	Rerun after every update

Big idea: automation costs more at the start but pays back through reuse.

1.3 Benefits of automation (slide 8)

Memory trick: **FIRRC**.

1. **F**aster execution (200 Daraz regression tests in minutes instead of hours).
2. **I**ncreased coverage (many browsers and screen sizes in one run).
3. **R**eusability (the same checkout script after every release).
4. **R**eliability (same steps every time, nothing skipped).
5. **C**ost-effective (a JazzCash transaction test written once, run hundreds of times).

1.4 Challenges of automation (slide 9)

1. **Initial setup time and cost** (a Selenium framework can take days before real testing starts).
2. **Maintenance overhead** (if a login button moves, every script that uses it must be updated).
3. **Not suitable for UI/UX feel** (a tool cannot judge whether a design is user-friendly; only a human can).
4. **Skilled workforce required** (Selenium needs Java or Python).
5. **False positives and negatives** (a test fails because of slow internet, not because of a real bug).

Challenge 3 is why manual testing never disappears. A brand-new feature whose look and feel must be judged is a manual job.

1.5 Automation workflow (slide 6)

This slide is a picture and could not be read from the file. The standard workflow **(extra)** is: choose test cases worth automating, choose a tool, set up the environment, write the scripts, run them, analyse the results, maintain the scripts. Compare with your slide.

1.6 Katalon Recorder

- A **lightweight browser extension** for **record-and-playback** testing. Works on Chrome, Firefox and Edge.
- You perform the steps once while it records; later it replays them. No coding needed.

- Scripts can be **exported** to Selenium WebDriver and Katalon Studio.
- **Install:** open Chrome or Firefox, go to the Web Store or Add-ons page, search for Katalon Recorder, click Add to Browser, then pin it.

Four main features (slide 13):

1. Record-and-playback.
2. **Assertions:** check that an element or page state is correct. This is how a test decides pass or fail.
3. Multiple export options (Selenium WebDriver, Katalon Studio).
4. **Test suites:** several test cases run together.

Test case vs test suite: a test case is one scenario (for example login). A test suite is a container that groups several test cases and runs them together (login plus search).

Slide 14 example, login on Automation Exercise: open the site, start recording, click Login, enter the email and password, submit, stop recording, save the test case. Only actions between Start and Stop are recorded.

Practice activities: Activity 1 records a product search. Activity 2 records login and product search as two test cases, puts them in one test suite, and plays the suite.

1.7 Reading a recorded test

Every row has three columns:

Column	Meaning	Example
Command	The action	open, click, type, select
Target	Which element (the locator)	id=my-text-id
Value	The data to use, if any	Arham

The Target is written as **type=value**: `id=my-text-id`, `name=my-password`, `css=input[name='my-password']`, `xpath=//button[@type='submit']`. No space is allowed after the `=`.

After playing a test, the **Log** panel shows each step and ends with “Test case passed”, and the counters show Passed 1, Failed 0.

Passed does not mean correct. A test with no assertion only proves that every step could be executed. It does not prove the result was right.

1.8 Commands used in class (extra: not on the slides)

The slides do not list commands. These are the standard Katalon Recorder ones, so compare with your class notes.

Group	Command	Target	Value
Action	open	URL	empty
Action	click	element	empty
Action	type	element	text to write
Action	sendKeys	element	keys, for example ${KEY_ENTER}
Action	select	dropdown	label=One
Action	check / uncheck	checkbox	empty
Verify	assertText / verifyText	element	expected text
Verify	assertTitle / verifyTitle	empty	expected title
Verify	assertElementPresent / verifyElementPresent	element	empty
Wait	pause	empty	milliseconds, for example 2000
Wait	waitForElementPresent	element	empty
Helper	echo	message	empty

type vs sendKeys: `type` sets the text directly in the box. `sendKeys` presses keys like a real keyboard, so it can also send special keys such as `${KEY_ENTER}` and `${KEY_TAB}`.

assert vs verify: a failed `assert` stops the test immediately. A failed `verify` records the failure and the test continues.

AndWait commands (clickAndWait, sendKeysAndWait): perform the action, then wait for the new page to load. Use them when the action opens a new page, such as submitting a form.

1.9 Self-healing (extra)

When you record, Katalon saves several alternative locators for each element. If your main locator breaks (for example `id=-galatid`), it tries the alternatives, finds the element, and the test still passes. The **Self-healing** tab shows the broken locator and the proposed

one. Danger: if the tool picks the wrong element, the test passes while a real problem exists (a false positive). To force a real failure, use a locator that was never recorded, or break the URL.

1.10 Debugging and limitations (slides 17 to 20)

Common errors: unrecognised elements or commands, and incorrect XPath or CSS locators. **Debugging tools:** pause, resume and step through; modify the recorded steps; read the Log.

Katalon Recorder limitation	Example	Selenium benefit
Limited scripting flexibility	No loops or conditions	Custom logic in Java or Python
Basic support for complex scenarios	File upload, multiple windows, pop-ups	Handles downloads and window switching
No parallel execution	Tests run one after another	Selenium Grid runs many browsers at once
Basic reporting	Few logs, no failure screenshots	Extent Reports with screenshots

Memory trick: **F-C-P-R** (Flexibility, Complex scenarios, Parallel, Reporting).

Conclusion (slide 21): Katalon Recorder is an easy tool for quick web testing; recording, exporting and playback are simple for beginners.

1.11 Week 1 exam traps

- Do not write that automation has no cost or replaces manual testing.
- A test that passes with no assertion has proved very little.
- Not every failed test is a bug: slow loading can cause a false failure.
- Typing `css= .x` or `css=. x` with a space is wrong. Write `css=.x`.

2. Week 2: Selenium WebDriver and Maven

2.1 What is Selenium?

- An automation tool used mainly for **web applications**.
- **Free** (no licensing cost).

- Runs on many operating systems, devices and browsers.
- Tests can be written in Java, Python, Kotlin, JavaScript and more.
- Example: automate a login (enter username and password, click Login).
- **Why Java?** Flexible, strong IDE support (IntelliJ), large community.

Katalon records tests; Selenium needs code. That removes Katalon's limitations but needs programming skill (Week 1 challenge 4).

2.2 What is Selenium WebDriver?

- The part of the Selenium suite that **interacts with web elements** (buttons, text boxes, links).
- **Platform-independent**: works across browsers.
- Provides a **programming interface** to create and run scripts that control browsers.
- Think of it as a remote control: your code gives commands and WebDriver makes the browser perform them.

Three key features (slides 6 to 8):

1. **Browser automation:** navigate and interact with a site.
2. **Cross-browser support:** run the same script on different browsers.
3. **Language support:** write the same test in Java or Python.

```
WebDriver driver = new ChromeDriver();      // open Chrome, create the remote
control
driver.get("https://www.daraz.pk");          // open the site
driver.findElement(By.linkText("Mobiles & Tablets")).click(); // find the
link, click it
```

`WebDriver` is the interface (the general remote control). `ChromeDriver` is the specific class that controls Chrome. `FirefoxDriver` controls Firefox. You need the matching browser driver (ChromeDriver, GeckoDriver for Firefox).

Cross-browser choice (slide 7):

```
WebDriver driver;
String browser = "firefox";
if (browser.equals("chrome")) {
    driver = new ChromeDriver();
} else if (browser.equals("firefox")) {
    driver = new FirefoxDriver();
}
driver.get("https://www.daraz.pk");
```

Why use WebDriver (slide 9): to check that web apps still work after updates, and to support continuous integration and delivery.

2.3 Maven

Maven is a **build automation and project management tool** for Java. You list the libraries you need in one file, **pom.xml**, and Maven downloads and manages them. Analogy: you hand a grocery list to the shop and the goods arrive.

Project folders:

Folder or file	Purpose
src/main/java	Application code (rarely needed in a test project)
src/test/java	Your test code
pom.xml	Dependencies

Dependencies from the handout (paste after `</properties>`, then click the Maven reload icon). Copied from mvnrepository.com:

```
<dependencies>
    <dependency>
        <groupId>org.seleniumhq.selenium</groupId>
        <artifactId>selenium-java</artifactId>
        <version>4.39.0</version>
        <scope>compile</scope>
    </dependency>
    <dependency>
        <groupId>org.testng</groupId>
        <artifactId>testng</artifactId>
        <version>7.8.0</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

Each dependency has groupId (library group), artifactId (library name), version and scope. Versions on your own machine may be newer; the method is the same.

2.4 TestNG and annotations

TestNG is a testing framework used to organise and run automated tests in a structured, reusable way. **Annotations** are metadata tags (starting with `@`) that control when a method runs.

Annotation	When it runs
@Test	Marks a method as a test case
@BeforeClass	Once, before any @Test in the class
@AfterClass	Once, after all @Test methods in the class
@BeforeMethod	Before each @Test method
@AfterMethod	After each @Test method

Class ones run once; Method ones run with every test. With two tests the order is:

```
@BeforeClass (once)
  @BeforeMethod -> @Test test1 -> @AfterMethod
  @BeforeMethod -> @Test test2 -> @AfterMethod
@AfterClass (once)
```

(extra) TestNG runs test methods in alphabetical order by default. Use `@Test(priority = 1)` to fix an order.

2.5 Setting up the project (handout steps)

1. Install IntelliJ IDEA (the JDK is optional because IntelliJ can install it).
2. File > New > Project > Maven; location on the lab desktop; optional Group ID `com.example` and Artifact ID `SeleniumTestProject`; click Create.
3. Add the dependencies to pom.xml and click the reload icon.
4. Right-click src/test/java, create package `tests`, then class `FirstTest`.
5. Right-click the class and Run it.

2.6 The first test (FirstTest)

```
package tests;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.annotations.AfterClass;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.Test;

public class FirstTest {
    WebDriver driver;                       // remote control declared, no
browser yet
```

```
    @BeforeClass
    public void setup() {
        driver = new ChromeDriver();        // opens Chrome
        driver.manage().window().maximize();
    }

    @Test
    public void testcase() {
        driver.get("https://mvnrepository.com/");
        System.out.println("The title of the page is: " + driver.getTitle());
    }

    @AfterClass
    public void teardown() {
        driver.quit();                      // closes all windows, ends the
session
    }
}
```

The handout also adds `Thread.sleep(3000)` between steps only so you can watch the browser. Order of events: browser opens first (@BeforeClass), the test runs in the middle (@Test), the browser closes last (@AfterClass). **Best practice:** always call `driver.quit()`.

2.7 Basic Selenium commands

Group	Command	What it does
Browser	`new ChromeDriver()`	Opens a new Chrome window
Browser	`driver.get(url)`	Opens a URL
Browser	`driver.close()`	Closes the **current** window only
Browser	`driver.quit()`	Closes **all** windows and ends the session
Page info	`getTitle()`	Returns the page title
Page info	`getCurrentUrl()`	Returns the current URL
Navigation	`navigate().to(url)`	Goes to another page
Navigation	`navigate().back()` / `.forward()` / `.refresh()`	History and reload

Group	Command	What it does
Element	`findElement(By...)`	Finds an element (most important)
Element	`click()`	Clicks it
Element	`sendKeys(text)`	Types text into it
Element	`getText()`	Returns visible text
Element	`getAttribute(name)`	Returns an attribute value, e.g. placeholder
Element	`isDisplayed()` / `isEnabled()`	Returns true or false
Window	`manage().window().maximize()` / `.minimize()`	Window size

Example from the handout (PakWheels):

```
driver.get("https://www.pakwheels.com/");
WebElement searchBox = driver.findElement(By.name("home-query"));
searchBox.sendKeys("toyota");
WebElement searchButton = driver.findElement(By.id("home-search-btn"));
searchButton.click();
```

close() vs quit() is a very likely question: `close()` ends one window; `quit()` ends the whole session. Clean-up code should use `quit()`.

The handout's helper method draws a red border around an element and pauses, so you can see what Selenium found:

```
public void highlightElement(WebElement element) throws InterruptedException
{
    JavascriptExecutor js = (JavascriptExecutor) driver;
    js.executeScript("arguments[0].style.border='6px solid red'", element);
    Thread.sleep(3000);
}
```

2.8 Review of your own code (First_Test)

What you did well: correct use of @BeforeClass, @Test and @AfterClass with `quit()`; `By.id`, `By.name` and `By.className`; navigation commands (`navigate().to`, `refresh`, `back`); window handling with `getWindowHandle()`, `switchTo().newWindow(WindowType.TAB)`, `close()` and `switchTo().window(original)`.

What to improve:

1. **No check at the end.** The test fills the form and submits but never verifies the result, so it passes even if the form is wrong. Add a check, for example compare `driver.getCurrentUrl()` with what you expect.
2. Use `click()` on the button, as in the slides, instead of `submit()`.
3. `sendKeys("Two")` on a dropdown works on that page but is a shortcut. The proper tool is the `Select` class **(extra)**.
4. If you switch @BeforeClass to @BeforeMethod, the browser opens once per test, so `driver.quit()` must also move to @AfterMethod.
5. Remove unused imports.

2.9 Week 2 exam traps

- Selenium is the whole suite; WebDriver is one component of it.
- Selenium reads the DOM; it does not see the page like a human.
- `@BeforeClass` is once; `@BeforeMethod` is before every test.
- `close()` is not `quit()`.
- After closing a tab, switch back with `switchTo().window(handle)` before using the driver.
- Declaring `WebDriver driver;` without `driver = new ChromeDriver();` causes a NullPointerException.
- Handout slips: it says Run ‘FirstClass’ but the class is FirstTest, and it uses # for comments in Java code, which must be `//`.

3. Week 3: The DOM and Web Elements (remaining)

3.1 What is the DOM?

DOM means **Document Object Model**. It is the browser’s structured, programmatic representation of an HTML page. The flow is:

HTML code → DOM → Web page

The browser reads the HTML and builds a **DOM tree**. HTML is the source; the DOM is the browser’s organised version of it. Selenium does not see the page like a human. **It interacts with the DOM.**

University-building analogy from the slide:

Building analogy	Web page meaning
Building	Web page
Rooms	HTML elements
Room numbers or names	Attributes such as id, name, class
Connections between rooms	Parent-child relationships

3.2 HTML elements and attributes

An **element** is a component of a web page. Take `<button id="loginBtn">Login</button>`:

Part	Value	Meaning
Tag (element type)	button	What kind of element it is
Attribute	id="loginBtn"	Extra information that helps identify it
Text or content	Login	The visible text
Complete element	opening tag + content + closing tag	

Common elements: `<input>`, `<button>`, `<a>`, `<h1>`, `<p>`, `<img>`, `<form>`, `<div>`.
Testers care about attributes because **attributes help us create locators**.

3.3 Why the DOM matters

A human thinks "click Login". Selenium needs to know which exact element, for example `driver.findElement(By.id("loginBtn")).click();`.

The DOM tells us what exists. Locators tell Selenium how to find it.

3.4 Inspecting an element (Developer Tools)

1. Open the page.
2. Right-click the element and choose **Inspect** (or press F12).
3. Find its HTML line in the Elements tab.
4. Note the tag, attributes (id, class, href, name) and text.
5. Ask: if I want Selenium to click this, what information can identify it?

3.5 How WebDriver talks to the DOM (slide 10)

- WebDriver and the browser DOM communicate both ways: WebDriver sends commands, the browser sends information back, through the browser-specific driver (for example ChromeDriver).
- Elements in the DOM are turned into **WebElement** objects and found with `findElement()`.
- `findElement()` takes a locator and searches the DOM until a match is found.

Slide correction: slide 10 says a null value is returned when nothing matches. In real Selenium, `findElement()` throws **NoSuchElementException**, and only `findElements()` returns an empty list. If a quiz answer key follows the slide, ask your teacher which wording she wants.

4. Week 3: Locators in Selenium (remaining)

A **locator** can be an ID, name, class name, tag name, link text, partial link text, CSS selector or XPath expression. It tells Selenium which element to find so it can click, type or select.

4.1 Locator types with Java syntax

Locator	Java syntax	Notes from the handout
ID	`By.id("sale_image")`	Fastest and most reliable; ids should be unique
Name	`By.name("email")`	Good for form fields (text boxes, radio buttons, checkboxes)
Class name	`By.className("features_items")`	Useful if the class is unique; fails for compound class names (with spaces)
Tag name	`By.tagName("input")`	Usually matches many elements; use with `findElements`
Link text	`By.linkText("Contact us")`	Only for `<a>` tags; exact visible text
Partial link text	`By.partialLinkText("art")`	Part of the link text; may match the wrong link if not unique

Locator	Java syntax	Notes from the handout
CSS selector	`By.cssSelector("input[name='email']")`	Flexible, attribute based
XPath	`By.xpath("//input[@id='email']")`	Powerful, but slower and more brittle

4.2 Which locator should I use?

Situation	Preferred locator
Unique id available	ID
Unique name available	Name
Unique class available	Class name
Hyperlink identified by visible text	Link text
Flexible attribute-based locator	CSS
Element has no useful unique attribute	XPath
Partial or dynamic matching	CSS or XPath
Relationship-based identification (parent, sibling)	XPath

Priority to remember: ID first, then Name, then Class name, then Link text, then CSS, and XPath when nothing else works.

4.3 findElement vs findElements

```
WebElement one = driver.findElement(By.id("email"));           // first
match, or exception
List<WebElement> all = driver.findElements(By.tagName("input")); // every
match, or empty list
System.out.println("Input fields count: " + all.size());
```

4.4 Examples from the handout (Automation Exercise)

Locator	Page	Element	Value
ID	products	Submit search	submit_search
ID	products	Sale image	sale_image

Locator	Page	Element	Value
Class name	home	Test cases button	test_cases_list
Name	contact_us	Name, Email, Subject inputs	name, email, subject
Tag name	contact_us	All input fields	input
Link text	home	Cart, Contact us	Cart, Contact us
Partial link text	home	Products	pro

The handout writes the class value as `test_case_list` in one place and `test_cases_list` in another. Always copy values from the live page.

4.5 Link text vs partial link text

- `linkText("Contact us")` needs the whole visible text, and works only on `<a>` tags.
- `partialLinkText("us")` needs only a part. Use it when the text changes, but make sure the part is unique: `art` could match more than one link.

5. Week 3: CSS and XPath (remaining)

5.1 CSS selectors

CSS selects elements by their attributes. In Katalon Target write `css=...`; in Selenium use `By.cssSelector("...")`; in the browser Ctrl+F box write only the selector.

Goal	Syntax	Example
By tag	tag	`input`
By id	#id	`#email`
By class	.class	`.form-control`
Attribute equals	tag[attr='v']	`input[name='userEmail']`
Attribute exists	tag[attr]	`input[name]`
Contains	[attr*='v']	`input[placeholder*='Email']`
Starts with	[attr^='v']	`input[placeholder^='Sub']`
Ends with	[attr$='v']	`textarea[placeholder$='Message']`
Multiple attributes	tag[a='1'][b='2']	`input[type='text'][id='email']`

For `<input type="text" id="email" name="userEmail" class="form-control">`: `#email`, `.form-control`, `input[name='userEmail']` and `input[id='email'][type='text']` all find it.

Signs to remember: `#` is id, `.` is class, `[ ]` is any attribute, `^` starts with, `$` ends with, `*` contains. A space means "anywhere inside" and `>` means "direct child".

5.2 XPath

XPath stands for **XML Path Language**. Basic form: `//tagname[@attribute='value']`. Read it as "anywhere on the page, find this tag whose attribute equals this value".

Level	XPath	Meaning
1 Basic	`//input`	All input elements
2 Attribute	`//input[@id='email']`	Input whose id is email
3 Multiple attributes	`//input[@type='text' and @name='email']`	Both conditions true
4 Text	`//button[text()='Login']`	Button with exact text Login
Any tag	`//*[@name='search']`	Any tag with that name
Contains	`//input[contains(@placeholder,'Name')]`	Partial attribute match
Contains text	`//h2[contains(text(),'Contact')]`	Partial text match
Starts with	`//h2[starts-with(text(),'Get')]`	Text starts with Get
Child	`//*[@class='productinfo text-center']/img`	Image inside that class

Why XPath exists: many elements have no id or name, and XPath can find elements by text, by position, or by relationship to a parent or sibling. It is powerful but slower and more brittle than other locators, so use it when nothing simpler works.

Nested paths: `//` means anywhere below, `/` means the direct child only. **(extra)** `following-sibling::input` goes to a neighbour element, and `parent::div` goes up to the parent.

5.3 Testing a locator before using it

1. Inspect the element, click in the Elements tab and press **Ctrl+F**.
2. Paste the XPath or CSS selector (no `xpath=` or `css=` prefix in the browser).
3. Read the counter: **1 of 1** is a perfect, unique locator. **0 of 0** means the locator is wrong. **1 of 5 or more** means it matches many elements and must be made more specific.

In Katalon the **Find** button on the Target row does the same job.

5.4 Locator mistakes that cost marks

- Using `input` in XPath for an element that is really a `textarea`. Check the tag in Inspect.
- A trailing space in text such as “Password ” makes `text()='Password'` fail. Use `contains(text(),'Password')`.
- Using `By.className` with a value containing spaces. Use CSS (`.a.b`) or XPath.
- Using `linkText` on something that is not an `<a>` tag.
- Writing XPath inside Java with double quotes inside double quotes. Use single quotes inside: `"//input[@id='email']"`.
- Handout slip: the “Add to cart” example uses `//a[text()='Add to cart']` under the label Button; the element is an `<a>` tag.
- Long absolute XPaths such as `//form/div[3]/div[2]/input` break when the layout changes. Prefer attribute-based ones.

6. Katalon vs Selenium at a glance

Task	Katalon Recorder	Selenium (Java)
Open a page	`open`	`driver.get(url)`
Find an element	Target `id=my-text-id`	`driver.findElement(By.id("my-text-id"))`
Type text	`type` or `sendKeys`	`sendKeys("text")`
Click	`click`	`click()`
Check a result	assert / verify commands	TestNG Assert (extra)
Wait	`pause` , `waitFor...`	Waits (extra, section 7)
Group tests	Test suite	TestNG class or XML suite

Task	Katalon Recorder	Selenium (Java)
Coding needed	No	Yes (Java)
Parallel runs, loops, reports	Limited	Supported

7. Preview of Weeks 4 to 7 (outline only)

You do not have slides for these weeks yet, so this is standard Selenium knowledge **(extra)**. Verify against your teacher's slides when they arrive.

Element interaction (Week 4): after locating an element you call methods on the WebElement: `sendKeys`, `click`, `getText`, `getAttribute`, `isDisplayed`, `isEnabled`, plus `clear()` and `isSelected()`. Dropdowns use the `Select` class (`selectByVisibleText`). A file input takes a full file path through `sendKeys`, then you click the real submit button.

Waits (Week 5): `Thread.sleep` waits a fixed time and is only for demos. An **implicit wait** is set once and applies to every `findElement`. An **explicit wait** (`WebDriverWait` with `ExpectedConditions`) waits for one condition and continues the moment it is true. Waits remove the false failures from slow loading.

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
WebElement btn =
wait.until(ExpectedConditions.elementToBeClickable(By.id("login")));
btn.click();
```

Actions class (Week 5): hover, double-click, right-click and drag and drop. Build the chain and finish with `.perform()`, or nothing happens.

Assertions (Week 6): an assertion compares actual with expected and fails the test if they differ. Order: actual, expected, message.

```
Assert.assertEquals(driver.getTitle(), "Automation Exercise", "Wrong title");
Assert.assertTrue(driver.findElement(By.id("sale_image")).isDisplayed());
```

A hard `Assert` stops the test at once; `SoftAssert` records failures and reports them at `assertAll()`. TestNG XML suites list classes to run together.

Page Object Model (Week 7): one class per page holds that page's locators and actions; tests hold only the steps and checks. If a locator changes you fix it in one place, which answers the maintenance-overhead challenge from Week 1.

8. Self-test: questions with answers

1. **Why is automation a good choice for re-checking a full ordering flow after every app update?** The task is repetitive and repeated every release; scripts are fast, reusable and never skip steps.
2. **Why should a brand-new page design be tested manually?** Judging whether a design is user-friendly needs a human (challenge: not suitable for UI/UX feel).
3. **Name the five benefits of automation.** Faster execution, increased coverage, reusability, reliability, cost-effective over many runs.
4. **Test case vs test suite?** A test case is one scenario; a test suite groups several test cases and runs them together.
5. **type vs sendKeys in Katalon?** `type` sets text directly; `sendKeys` presses keys like a keyboard and can send special keys such as Enter.
6. **assert vs verify?** A failed assert stops the test; a failed verify records the failure and continues.
7. **What does sendKeysAndWait do?** It sends the keys, then waits for the page to load.
8. **Why did a test with a wrong locator still pass in Katalon?** Self-healing found the element using a saved alternative locator.
9. **Name two Katalon Recorder limitations and the Selenium fix.** No parallel execution (Selenium Grid); basic reporting (Extent Reports); also limited logic and weak complex scenarios.
10. **What is Maven and what does pom.xml hold?** A build and project management tool; pom.xml lists the dependencies (Selenium, TestNG) that Maven downloads.
11. **When does each annotation run?** BeforeClass and AfterClass once per class; BeforeMethod and AfterMethod around every test; Test marks the test.
12. **A class has two @Test methods and one @BeforeMethod. How many times does it run?** Twice, once before each test. A @BeforeClass would run once.
13. **close() vs quit()?** `close()` closes the current window; `quit()` closes all windows and ends the session.
14. **What is the DOM?** The browser's tree representation of the HTML page, which Selenium works on.
15. **What do DOM and locators each do?** The DOM tells what exists; locators tell Selenium how to find it.
16. **Which locator first and why?** ID, because it is meant to be unique, fast and reliable.
17. **Why use XPath?** When there is no unique id or name, or when you need text or parent-child relations.
18. **Ctrl+F shows 0 of 0 for your XPath. What does it mean?** The locator matches nothing and is wrong; check the tag and the attribute value.

19. **Write a CSS selector for `<input name="my-password">`.** `input[name='my-password']` (Katalon: `css=input[name='my-password']`).
20. **Write an XPath for a submit button.** `//button[@type='submit']`.

9. Last-night revision checklist

- Manual vs automation table, FIRRC benefits, the five challenges.
- Katalon: four features, Command/Target/Value, test case vs suite, F-C-P-R limitations.
- Selenium vs WebDriver, three WebDriver features, ChromeDriver vs FirefoxDriver.
- Maven folders and pom.xml, the five annotations and their order, close vs quit.
- Basic commands table (get, findElement, click, sendKeys, getText, getAttribute, isDisplayed, navigate).
- DOM analogy, element parts (tag, attribute, text), Inspect steps.
- Locator table, locator priority, CSS signs (# . [ ] ^ $ *), XPath levels 1 to 4, Ctrl+F counter meaning.
- Know the slide-vs-reality point: `findElement` throws an exception when nothing matches.